

Prompt Engineering Single-Scenario Study

Comparing Naïve vs. Refined Prompting in Technical Concept Explanation

1. INTRODUCTION & OBJECTIVE

This experiment evaluates the operational differences between **Naïve Prompts** (short, minimal instructions) and **Basic/Refined Prompts** (structured, context-rich instructions) when applied to ChatGPT. The target scenario chosen for deep analysis is **Technical Concept Explanation**.

2. METHODOLOGY

The model was tasked with explaining a fundamental computer science concept: *Recursion*. Responses were evaluated across three core dimensions on a 1–5 scale:

- **Quality (1–5):** Clarity, coherence, formatting, and structural flow.
- **Accuracy (1–5):** Factuality, precision, and absence of technical errors.
- **Depth (1–5):** Thoroughness, technical nuance, code mechanics, and educational value.

3. SINGLE-SCENARIO EXPERIMENT TRIAL

Trial A: Naïve Prompt

Prompt: "What is recursion?"

Generated Response:

"Recursion is a programming concept where a function calls itself to solve a smaller instance of the same problem. A recursive function must have a base case to stop calling itself; otherwise, it will result in an infinite loop or stack overflow error."

Quality: 4.0 | Accuracy: 5.0 | Depth: 3.0 | Average: 4.00

Evaluation Note: High factual precision, but minimal depth. It defines the term accurately but lacks implementation details, code syntax, or execution stack analysis.

Trial B: Basic / Refined Prompt

Prompt: "Explain recursion to an undergraduate computer science student. Include a breakdown of base cases vs. recursive steps, a brief Python code example calculating a factorial, and an explanation of how the call stack handles the execution."

Generated Response:

Recursion is a method of solving a problem where the solution depends on solutions to smaller instances of the same problem.

1. Key Structural Components:

- **Base Case:** The terminating condition that stops recursion and returns a value without making further recursive calls.
- **Recursive Step:** The logic that reduces the problem size and invokes the function again.

2. Python Implementation:

```
def factorial(n):  
    if n <= 1: return 1 # Base Case  
    return n * factorial(n - 1) # Recursive Step
```

3. Call Stack Mechanics:

When factorial(3) is called, execution frames are pushed onto the call stack:

1. factorial(3) calls factorial(2)
2. factorial(2) calls factorial(1)
3. factorial(1) hits the base case and returns 1.

The stack then unwinds, evaluating values in reverse order (1 → 2 → 6).

Quality: 5.0 | Accuracy: 5.0 | Depth: 4.8 | Average: 4.93

Evaluation Note: Outstanding structure and completeness. Tailored effectively for an undergraduate audience with concrete syntax and step-by-step stack frame unwinding.

4. QUANTITATIVE SCORING SUMMARY

Scenario	Prompt Type	Quality (1–5)	Accuracy (1–5)	Depth (1–5)	Overall Avg
Technical Explanation	Naïve Prompt	4.0	5.0	3.0	4.00
Technical Explanation	Refined Prompt	5.0	5.0	4.8	4.93

Scenario	Naïve Avg. Score	Basic/Refined Avg. Score	Absolute Improvement
Technical Concept Explanation	4.00	4.93	+0.93

5. KEY FINDINGS & ANALYSIS

1. Accuracy Stability vs. Depth Expansion: Both prompt styles yielded 5.0 accuracy, demonstrating that LLMs maintain strong baseline truthfulness for core technical definitions. However, depth increased by +1.8 points when explicit constraints were provided.

2. Role & Audience Context: Setting the persona/audience ("undergraduate computer science student") naturally adjusted the explanation's complexity, encouraging the model to include underlying system concepts like memory frames and stack unwinding.

3. Multi-Modal Representation: Demanding prose, syntax (Python), and execution traces within a single prompt produces a far more effective educational artifact than open-ended queries.

6. CONCLUSION

For simple technical definitions, a naïve prompt is sufficient for quick reference. However, when the goal is comprehensive instruction or structured understanding, a refined prompt providing explicit context, components, and code syntax delivers significantly higher educational value (+0.93 overall improvement).